



DPUSGEVideoObject

Creates a simple overlay object in the virtual world that can be used to display a video texture.

1. Inputs:

<i>X,Y,Z</i>	Object's position in the simulation's world coordinate system [m]. Default: (0, 0, 0)
<i>rX,rY,rZ</i>	Object rotation around the x, y, and z axes [rad]. Default: (0, 0, 0)
<i>sX,sY,sZ</i>	Object scaling along the x, y, and z axes. Default: (1, 1, 1) (original object size)
<i>ColR, ColG, ColB, ColA</i>	Object color. Default: white (1, 1, 1, 1).
<i>Fog</i>	Influence of fog on the object. Default: 1 (normal influence of fog) Lower positive values improve the object's visibility in fog conditions.
<i>Show</i>	= 1 (default value): The object is visible. = 0: The object is (currently) invisible.

2. Inputs for each Parameter:

<i>Parami.X, Parami.Y, Parami.Z, Parami.W</i>	Coordinates X, Y, Z and W of the parameter value #i.
---	--

3. Static parameters:

<i>TextureName</i>	Name of the displayed SGE video texture (see "Functional principle"). The name must start with a dollar sign. If the video texture doesn't exist, a white rectangle will be displayed instead. Default value: <i>\$Video</i>
<i>Transparent</i>	If set to <i>true</i> , transparent parts in the video texture are blended onto the scenery. If set to <i>false</i> (the default), transparency in the video texture is ignored. Note: Most video sources never output transparent pixels.
<i>VertexShader</i>	Vertex shader name.
<i>PixelShader</i>	Pixel shader name.
<i>Visibility</i>	Visibility category for the object. It can be used to display the object on one of the views updated on a single graphics computer only.
<i>NMaxParams</i>	Number of shader parameters (0..10, default: 0)



4. Dependencies to other DPUs:

Please make sure that the computer on which **DPUSGEVideoObject** runs also executes **DPUSGEWorld**, **DPUSGETextureXXX** and **DPUSGEView**. The Index values of these DPUs must be set in the following order:

`DPUSGEWorld.Index < DPUSGETextureXXX.Index < DPUSGEVideoObject.Index < DPUSGEView.Index`

(The name `DPUSGETextureXXX` stands for any of the video texture DPUs, such as `DPUSGETextureDirectShow` or `DPUSGETexturePylon`.)

5. Functional principle:

Using *DPUSGEVideoObject*, an additional graphical 2D object is created for the simulation. It is suitable to display a dynamic video texture as created by one of the video texture DPUs, such as *DPUSGETextureDirectShow* or *DPUSGETexturePylon*. *DPUSGEVideoObject* creates a simple 2D video overlay object and fills it with the video texture. The overlay object is located at the origin of the overlay coordinate system (i.e. the top-left edge of the *SGEView*) and its size is 1x1 pixels. The position and size of the overlay object can be defined by setting the inputs *X*, *Y*, *Z* (position) and *sX*, *sY* (size in pixels). By default, a simple pixel shader is used which just uses the video's colors. Optionally, different pixel and vertex shaders can be used by specifying the *PixelShader* and/or *VertexShader* parameters.

The following DPU configuration section displays the video texture *\$MyVideo* on a rectangular image at position (*X*=10, *Y*=0) and size (*X*=400, *Y* = 300) pixels in the simulation:

```
DPUSGEVideoObject TestObject
{
    Computer = {XYZ};
    Index = 60;
    TextureName = "$MyVideo";
    X = 10.0;
    Y = 0.0;
    Z = 8.0;
    sX = 400.0;
    sY = 300.0;
    Show = 1;

    NMaxParams = 2;
};
```

6. Shader parameters:

If the included object uses a special, user-defined vertex or pixel shader, parameters can be passed from DPU inputs directly to the shader.

To do so, the static DPU parameter *NMaxParams* must be used to create the re-quired amount of shader parameter inputs:



```
DPUSGEVideoObject TestObject
{
    Computer = {XYZ};
    Index = 60;
    TextureName = "$MyVideo";
    X = 10.0;
    Y = 0.0;
    Z = 8.0;
    sX = 400.0;
    sY = 300.0;
    Show = 1;

    NMaxParams = 2;
};
```

Each of these parameters is created as a vector of four components and thus has the “coordinates” X, Y, Z and W. However, these values are passed to the shader without any modification. Consequently, they can also be used to pass four unrelated floating-point values.

Default values for shader parameters can be assigned in the DPU section. Of course, the shader parameters can also be connected within the SILAB system in order to modify the values dynamically.

```
DPUSGEObject TestObject
{
    Computer = {XYZ};
    Index = 60;
    Object = "models.MyShaderObject";
    X = 10.0;
    Y = 0.0;
    Z = 0.0;
    Show = 1;

    NMaxParams = 2;

    Param1.X = 10.0;
    Param1.Z = 231.29;
    Param2.X = 13;
};

Connections = {
    MyOscillator.Out <-> TestObject.Param1.Y
};
```

Within the shader, these parameters must be declared as the variable *uniform vec4 Parami*; they can then be used to perform arbitrary calculations.